

Data Structure

1. DS & ADT

1주차: 자료구조 & 추상 자료형

? 오늘의 핵심 질문

동일한 데이터 100만 개가 있을 때, 특정 값을 찾거나 추가하는 속도는
왜 그릇의 모양에 따라 천차만별로 달라질까?

이번 시간에 해결할 주요 의문점들:

→ 저장 vs 연산

데이터를 그냥 쌓아두는 것과 자주 쓰는 연산에 맞춰
조직하는 것의 차이점은 무엇인가?

→ ADT와 구현체

개념적인 설계도(ADT)와 실제 파이썬 코드로 구현된 구조는
어떻게 연결되는가?

목차 INDEX

01 자료구조

- 자료구조란 무엇인가
- 자료구조가 필요한 이유
- 데이터와 자료구조의 차이
- 연산 중심 사고
- 주요 연산

02 ADT

- ADT
- 구현체
- 리스트 VS 파이썬의 리스트
- 실제 RAM의 메모리 구조

03 복잡도

- 시간복잡도
- 빅-오 표기법
- 공간복잡도
- 파이썬 내부 자료형 개요
- 시퀀스 성능 비교

01

자료구조

Data Structure

자료구조란 무엇인가?

자료구조(Data Structure)는 데이터를 컴퓨터 메모리 내에
저장하고, 접근하고, 수정하는 방법을 체계적으로 정한 구조이다.

단순한 "그릇"이 아니다

데이터를 단순히 담아두기만 하는 물리적 상자가 아니라, 데이터를 어떻게 조직(Organize)할 것인가에 대한 설계

저장 방식과 연산 비용

저장된 형태에 따라 특정 연산(Search, Insert) 수행 시
소요되는 비용(시간·메모리)이 완전히 결정

자료구조가 필요한 이유

1. 탐색 및 확인

특정 값 찾기, 중복 여부 확인

2. 데이터 변경

값 하나 추가하기, 값 하나 삭제하기

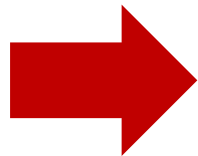
3. 출력 및 순회

순서대로 정렬하여 출력하기

자료구조가 필요한 이유

백만개의 데이터에서 999,999 라는 값을 찾는 실험

자료형	수행한 연산	소요 시간
list	999999 in data	약 6.24 ms (약 6240 μ s)
tuple	999999 in data	약 4.68 ms (약 4680 μ s)
dict	999999 in data	약 0.032 μ s
set	999999 in data	약 0.028 μ s



list와 set는 약 **221,429배**의 성능 차이를 보임

💡핵심 의문: 데이터를 어떤 구조(list, set, dict 등)에 저장하느냐에 따라 왜 실행 속도와 메모리 효율이 극단적으로 달라질까요?

“데이터”와 “자료구조”의 차이

데이터 (Data)

저장하려는 **값 자체** (예: 10, 20, 30, 40)

값 자체는 변하지 않지만, 이를 컴퓨터가 인식하고 다루는 모습은 다를 수 있습니다.

자료구조 (Data Structure)

데이터를 **조직하고 다루는 방식**

같은 값이라도 어떤 구조로 엮느냐에 따라 성능과 기능이 바뀝니다.

동일한 데이터(10, 20, 30, 40)를 파이썬의 서로 다른 구조로 표현:

```
numbers_list = [10, 20, 30, 40]
numbers_tuple = (10, 20, 30, 40)
numbers_set = {10, 20, 30, 40}
numbers_dict = {"a": 10, "b": 20, "c": 30, "d": 40}
```

list

tuple

set

dict

자료구조와 알고리즘의 관계

데이터

저장할 원본



자료구조

데이터 저장 조직



알고리즘

데이터 처리 절차



결과 및 성능

속도와 효율성

✓ 구조와 절차의 협업

자료구조는 '어떻게 담을 것인가', 알고리즘은
'어떻게 처리할 것인가'를 담당

⚠ 부적절한 조합의 위험

아무리 훌륭한 알고리즘을 짜도 부적절한
자료구조를 선택하면 연산 속도가 심각하게 느려짐

연산 중심 사고

"데이터의 모양보다 자주 수행할 연산을 먼저 본다."

자료구조를 선택하기 전 던져야 할 필수 질문들:

1. 규모와 순서

데이터가 얼마나 많은가?
순서를 유지해야 하는가?

2. 중복과 접근

중복을 허용하는가?
인덱스로 빠르게 접근하는가?

3. 탐색과 수정

검색을 자주 하는가?
삽입/삭제가 빈번한가?

주요 연산

접근 / 조회 (Access)

특정 위치의 데이터를 바로 가져오기

```
value = data[3] # 인덱스로 접근
```

탐색 (Search)

특정 값이 존재하는지 찾기

```
found = (target in data)
```

삽입 (Insert)

새로운 데이터 추가하기

```
data.append(10) # 끝에 추가
```

삭제 (Delete)

기존 데이터 제거하기

```
data.remove(10)
```

순회 (Traversal)

모든 데이터를 빠짐없이 방문

```
for x in data:  
    print(x)
```

중복 확인

동일한 값이 있는지 검사

```
# 집합 등을 활용
```

02

추상 자료형

Abstract Data Type

추상 자료형 Abstract Data Types

추상 자료형(ADT)은 어떤 데이터를 저장하고, 어떤 연산을 제공하는지를 정의한 **개념적 설계도**이다.

'무엇'을 하는가에 집중

내부 메모리를 어떻게 조작하는지 상세한 구현 방식은 숨기고,
외부 사용자(개발자)가 쓸 수 있는 기능 목록만 정의

설계도와 건물의 차이

ADT는 건축 설계도라면, 실제 자료구조 구현체는
그 설계도를 바탕으로 벽돌과 콘크리트로 지은 실제 건물

추상 자료형의 예시: Stack

↑ **push(item)**

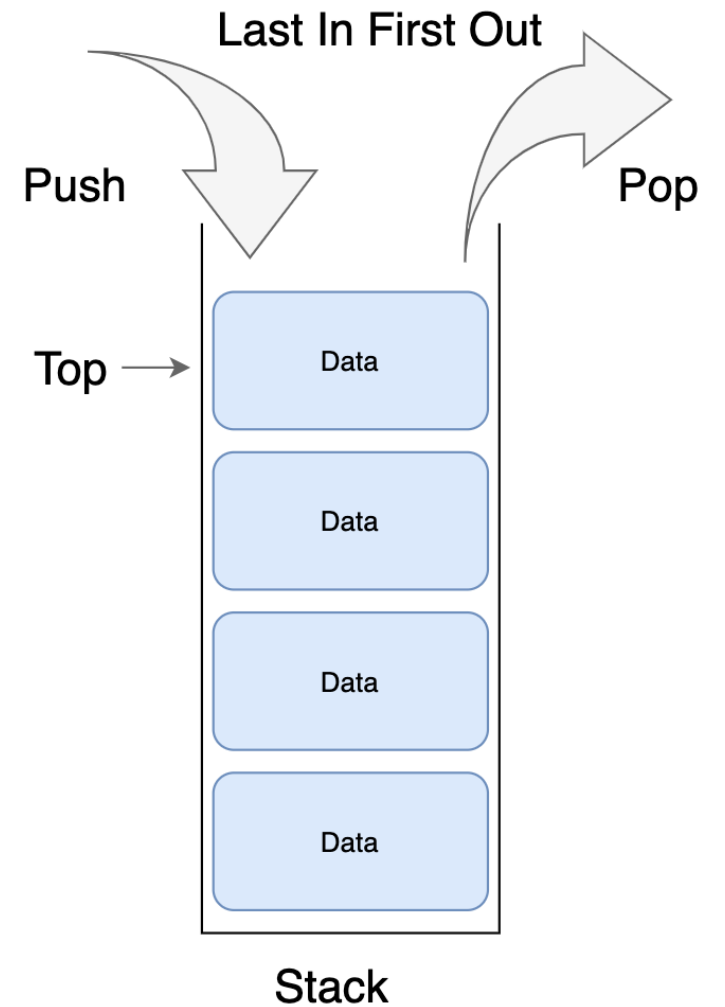
스택 맨 위에 새로운 데이터 추가

↓ **pop()**

맨 위에 있는 데이터 제거 후 반환

🕒 **peek() / is_empty()**

맨 위 확인 / 비었는지 검사



02 추상 자료형 Abstract Data Types

구현체 Implementation

추상 자료형 (Stack ADT)

제공 기능: push(), pop(), peek()



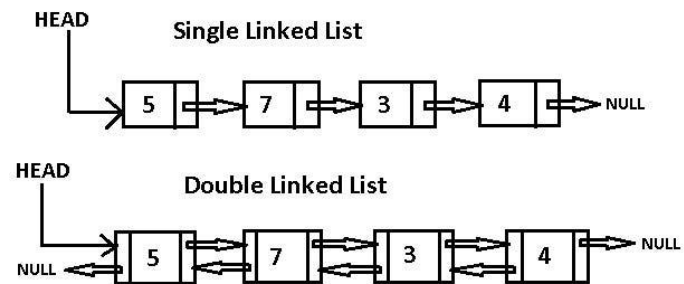
구현체 A: Python list

파이썬의 동적 배열 기반으로 스택 연산을 내부적으로 구현

```
numbers_list = [10, 20, 30, 40]
```

구현체 B: 연결 리스트 (Linked List)

노드 체인을 활용하여 메모리를 연결해 스택 연산을 구현
(나중에 학습)



Array vs List



대다수 언어에서는 list 자료형을 array라 부름

■ 리스트(List) ADT

순서가 있는 데이터의 모임을 뜻하는 추상적 개념
중간에 값을 삽입하거나 삭제하는 연산 규칙 정의

⚙ CPython의 list 구현 (다른 언어들의 Array에 해당)

파이썬(CPython) 내부에서 list는 연결 리스트가 아니라,
연속된 메모리 공간에 객체들의 참조(주소)를 저장하는
동적 배열(Dynamic Array)에 가까움

[참고] 각 자료형들의 크기

- 메모리 크기는 몇 자리 이진수로 저장되는지를 나타냄
- 1byte = 8bit (이진수 8개)
- 그래서 int는 $2^{(4 \times 8)}$ 개의 값을 가질 수 있음
- 박스 이외의 자료형들은 C언어의 자료형 (잘 안씀)
- 파이썬에서는 int가 정수형 자료형을 모두 포함하나, C를 비롯한 대다수의 프로그래밍 언어에서는 정수형 자료형들을 크기에 따라 구분함

구분	자료형	메모리크기 (byte)	범위
문자형	char	1	-128 ~ + 127
정수형	short	2	- 32768 ~ + 32767
	int	4	- 2147483648 ~ + 2147483647
	long	4	- 2147483648 ~ + 2147483647
실수형	float	4	$\pm 3.4 \times 10^{-37} \sim \pm 3.4 \times 10^{38}$
	double	8	$\pm 1.7 \times 10^{-307} \sim \pm 1.7 \times 10^{308}$
	long double	8	double 이상

실제 RAM의 메모리주소 구조

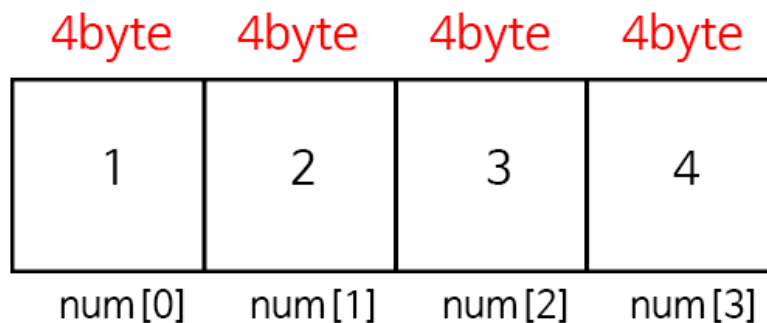
- 실제 컴퓨터의 RAM 주소는 이런 구조임
- 16진수로 된 주소 번호와 그 안에 들어가는 값으로 구성
- 32비트 컴퓨터에서는 한 칸이 4바이트,
64비트 컴퓨터에서는 한 칸이 8바이트임
옆의 그림은 간략하게 나타낸 도식이라 한 칸에 1바이트(8비트)임

Address	Value
0x00	01001010
0x01	10111010
0x02	01011111
0x03	00100100
0x04	01000100
0x05	10100000
0x06	01110100
0x07	01101111
0x08	10111011
...	...
0xFE	11011110
0xFF	10111011

[참고] C Array의 메모리 구조

- C에서는 배열의 각 원소를 한 칸씩 메모리에 저장함
- 아래는 한 칸이 4바이트인 32비트 컴퓨터에서 각 칸에 int 값들을 저장한 모습임
- C는 배열에 하나의 자료형만 들어갈 수 있기 때문에 이렇게 해도 문제없음
- 아래와 같이 선언함

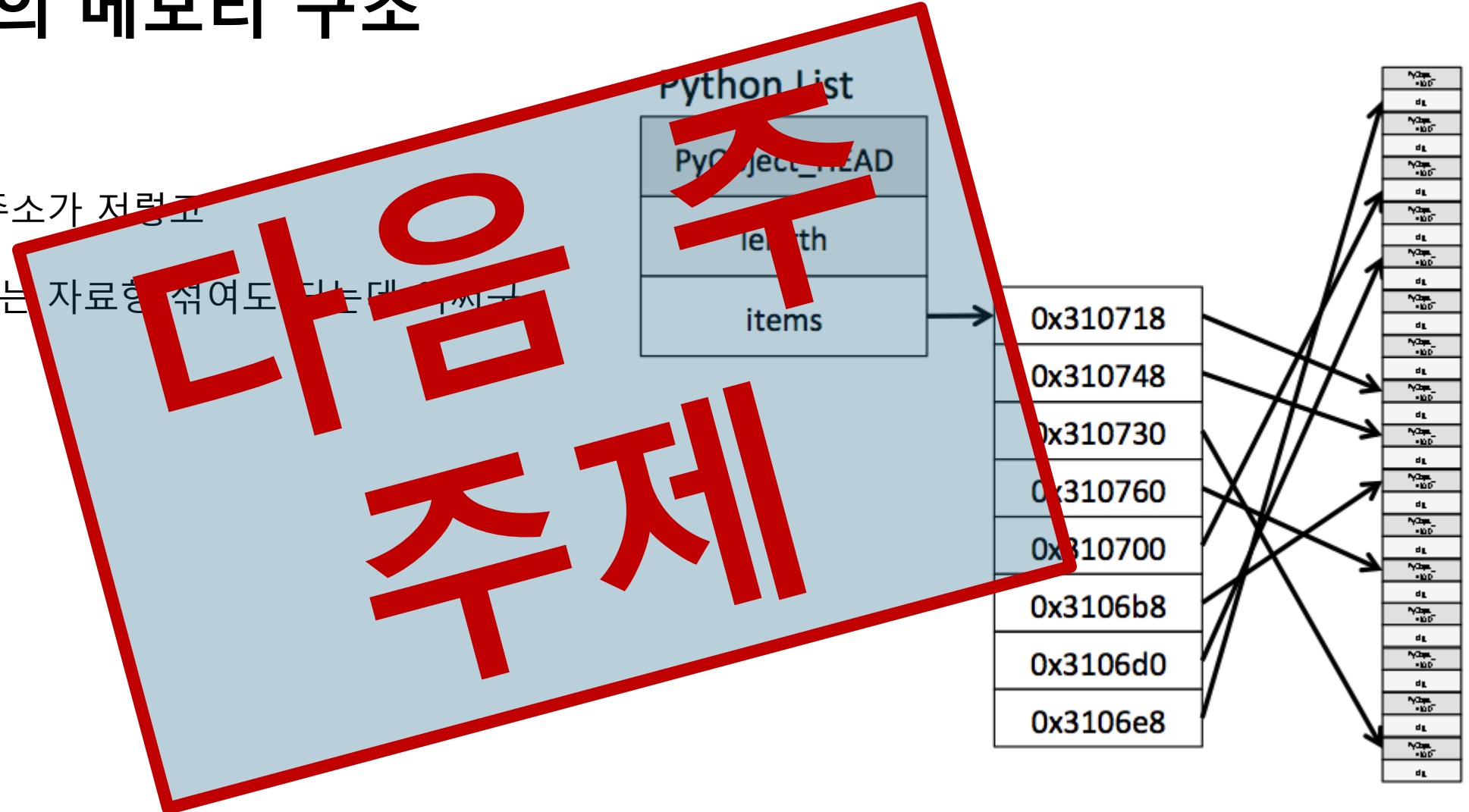
```
int num[4] = {1, 2, 3, 4};
```



Address	Value
0x00	01001010
0x01	10111010
0x02	01011111
0x03	00100100
0x04	01000100
0x05	10100000
0x06	01110100
0x07	01101111
0x08	10111011
...	...
0xFE	11011110
0xFF	10111011

Python List의 메모리 구조

- 객체가 어찌구
- 참조가 이렇고 주소가 저렇고
- 파이썬 리스트에는 자료형 섞여도 되는대 어찌구



03

복잡도

Complexity

시간복잡도 Time Complexity

시간복잡도는 입력 크기(**N**)가 커질 때 연산 횟수가 **어떤 속도로 증가하는지** 나타내는 수학적 표기법이다.

🕒 절대적 시간이 아니다

컴퓨터의 CPU 성능, 운영체제에 따라
초 단위 실행 시간은 달라짐
따라서 하드웨어와 무관하게 **증가 경향** 비교

ㄴ 빅오(Big-O) 표기법

가장 빠르게 증가하는 항(최고차항)만 남겨서 표현
(예: $O(1)$, $O(N)$, $O(N^2)$)

빅-오 표기법 Big-O Notation

$O(1)$ 상수 시간

데이터 크기와 무관하게
항상 일정한 연산 횟수

```
value = data[3] # 인덱스 접근
```

$O(\log N)$ 로그 시간

데이터가 늘어날 때마다
탐색 범위가 절반씩 감소

```
# 이진 탐색 (정렬된 데이터)
```

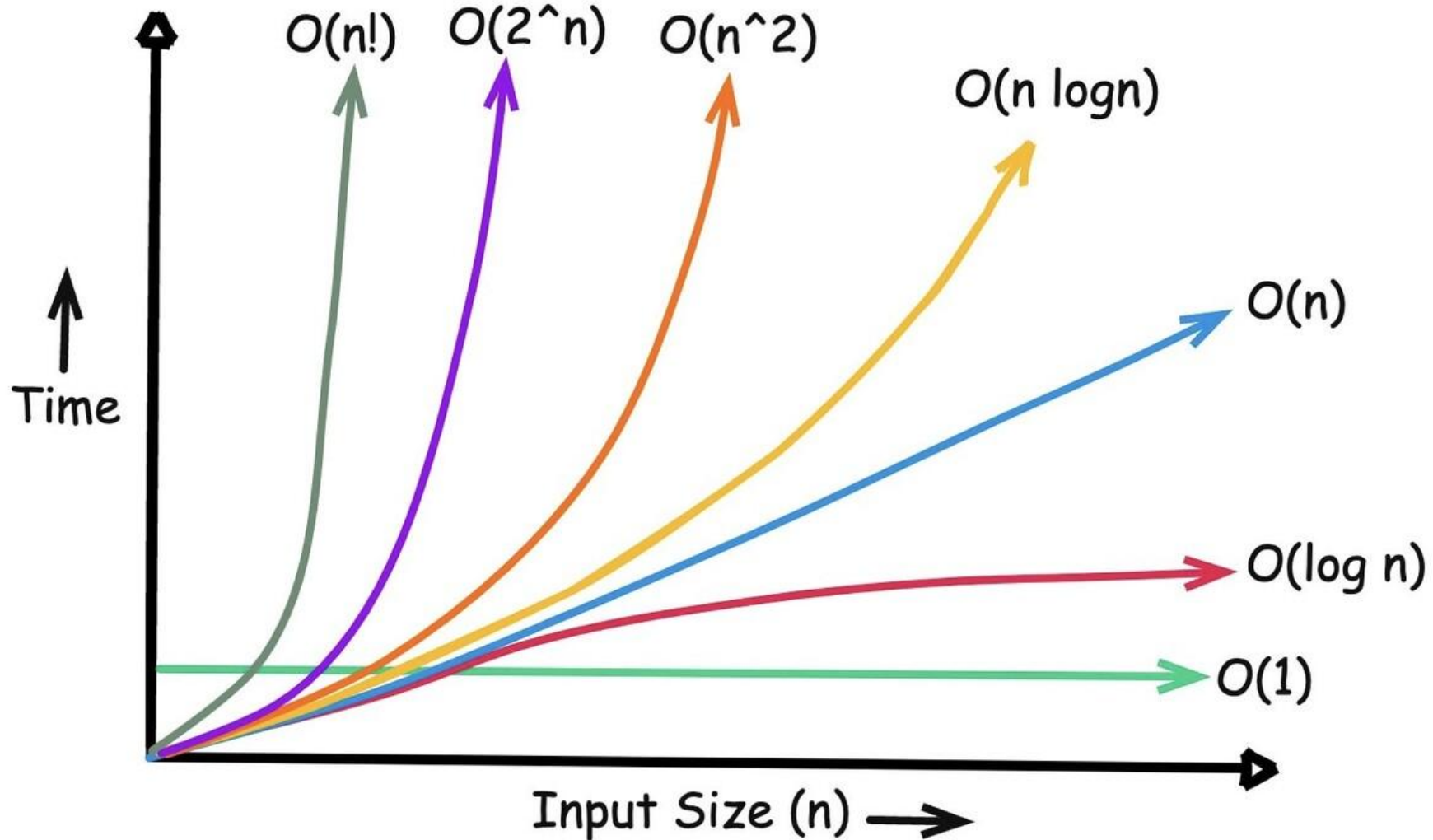
$O(N)$ 선형 시간

데이터 크기(N)에 비례하여
연산 횟수 증가

```
target in data_list # 순차 탐색
```

⚠ $O(1)$ 은 무조건 단 1번의 CPU 연산만 실행된다는 뜻이 아닌 입력 크기와 무관하게 연산 횟수가 일정하단 뜻

빅-오 표기법 Big-O Notation



03 복잡도 Complexity

공간복잡도 Space Complexity

공간복잡도는 입력 크기가 커질 때 프로그램이 필요로 하는 **추가 메모리(보조 공간, Auxiliary Space)**의 증가량을 나타낸다.

↻ 제자리 처리 (In-place)

```
# 추가 메모리 거의 안 씀  $O(1)$ 
for i in range(len(data)):
    data[i] *= 2
```

+ 새 리스트 생성 (New Allocation)

```
# 새로운 리스트 공간 할당  $O(N)$ 
result = [x * 2 for x in data]
```

⚠ 여기서의 빅-오 표기법은 시간 증가가 아닌 필요한 메모리 증가를 뜻함

파이썬 내부 자료형 개요

1. list (리스트)

순서 있음, 중복 허용, 수정 가능, 인덱스 접근 빠름

2. tuple (튜플)

순서 있음, 중복 허용, 수정 불가능, 고정 데이터 표현에 적합

3. set (셋/집합)

순서 없음, 중복 허용 안 함, 빠른 멤버십 검사(평균 $O(1)$)

4. dict (딕셔너리)

키-값(Key-Value) 쌍 매핑, 키를 통한 초고속 조회

파이썬 내부 자료형 개요

구분	순서 유지	중복 허용	수정 가능 여부	인덱스 접근	대표 활용 사례
list	O	O	변경 가능 (Mutable)	O	순서 있는 작업 목록, 순차 데이터
tuple	O	O	변경 불가 (Immutable)	O	고정 좌표, 함수의 다중 반환값
set	X	X	변경 가능 (Mutable)	X	중복 제거, 빠른 존재 여부 검사
dict	O (3.7+)	Key X / Val O	변경 가능 (Mutable)	Key로 접근	학생 이름과 점수 매핑, 설정값 저장